

EFTA00400459 BASE64 Decoded - Instructions to replicate

Epstein DOJ Dataset 9 — Base64 PDF Attachment Successfully Decoded

TL;DR

The embedded PDF attachment in EFTA00400459.pdf has been fully recovered. It is a 2-page charity gala invitation for the **Dubin Breast Center Second Annual Benefit**, held Monday, December 10, 2012, at the Mandarin Oriental in New York City. 39 of 40 FlateDecode streams were successfully decompressed and all text content was extracted.

Document Metadata

Field	Value
Filename	DBC12 One Page Invite with Reply.pdf
MIME Content-Type	application/pdf; name="DBC12 One Page Invite with Reply.pdf"
Content-Transfer-Encoding	base64
Expected Size	276,028 bytes (per MIME Content-Length)
Recovered Size	275,971 bytes (per-line decode from KoKuToru OCR)
PDF Version	1.5
Creator	Adobe Illustrator CS4 (v14.0)
Producer	Adobe PDF library 9.00
Creation Date	November 8, 2012, 12:40:09 PM
Modification Date	November 8, 2012, 12:40:10 PM
Title	Basic CMYK
Working Filename	DBC12_einvitation_rsvp.pdf
Pages	2
Fonts	Gotham-Medium, Archer-BoldSC, Archer-Medium, Avenir-Book, Avenir-Roman, Wingdings
Color Space	CMYK with PANTONE 225 C (hot pink/magenta), PANTONE 541 M (navy blue)
Created By	Karen Hsu (per XMP metadata)

Source Email Context

Field	Value
Source Document	EFTA00400459.pdf
Dataset	Epstein DOJ Dataset 9
Source PDF Size	11.25 MB, 76 pages
Email Date	December 3, 2012
Email Domain	cpusers.carillon.local
Associated Name	Boris Nikolic
Base64 Lines	4,843 lines at 76 chars each
MIME Boundary	Present at line 4853

Recovered Content

PAGE 1 — INVITATION

PLEASE JOIN

BENEFIT CO-CHAIRS GABRIELLE AND LOUIS BACON ALEXANDRA AND STEVEN COHEN EVA AND GLENN DUBIN AMY AND JOHN GRIFFIN WENDY HAKIM JAFFE SONIA AND PAUL TUDOR JONES II ALLISON AND HOWARD LUTNICK VERONIQUE AND BOB PITTMAN BETH AND DAVID SHAW KATHLEEN AND KENNETH TROPIN NINA AND GARY WEXLER JILL AND PAUL YABLON

FOR THE

DUBIN BREAST CENTER

SECOND ANNUAL BENEFIT

MONDAY, DECEMBER 10, 2012

HONORING **ELISA PORT, MD, FACS AND THE RUTTENBERG FAMILY**

HOST **CYNTHIA MCFADDEN**

SPECIAL MUSICAL PERFORMANCES **CAROLINE JONES, K'NAAN, HALEY REINHART, THALIA, EMILY WARREN**

MANDARIN ORIENTAL 7:00PM COCKTAILS · LOBBY LOUNGE 8:00PM DINNER AND ENTERTAINMENT · MANDARIN BALLROOM *FESTIVE ATTIRE*

PAGE 2 — REPLY / RSVP CARD

DUBIN BREAST CENTER SECOND ANNUAL BENEFIT MONDAY, DECEMBER 10, 2012 *HONORING* ELISA PORT, MD, FACS AND THE RUTTENBERG FAMILY MANDARIN ORIENTAL, NEW YORK CITY

PLEASE ADD MY NAME TO THE BENEFIT COMMITTEE AND RESERVE THE FOLLOWING:

	Tier	Price	Benefits
☐	ONE PLACE TABLE	\$100,000	Table for 10, priority seating, special recognition, One Place listing in printed program, listing on Annual and Permanent Donor Walls, Diamond Circle benefits of the Circle of Friends
☐	ONE MISSION TABLE	\$50,000	Table for 10, premium seating, special recognition, One Mission listing in printed program, listing on Annual Donor Wall, Platinum Circle benefits of the Circle of Friends
☐	ONE TEAM TABLE	\$25,000	Table for 10, excellent seating, One Team listing in printed program, listing on Annual Donor Wall, Gold Circle benefits of the Circle of Friends
☐	ONE PURPOSE TABLE	\$10,000	Table for 10, One Purpose listing in printed program, listing on Annual Donor Wall, Silver Circle benefits of the Circle of Friends
☐	ONE ROOF TICKET(S)	\$2,500	Priority seating for dinner, One Roof listing in printed program
☐	ONE TICKET(S)	\$1,000	Seating for dinner, One listing in printed program

Please make checks payable to **Dubin Breast Center** (Tax-ID# 13-6171197) Return to **Event Associates, Inc.**, 162 West 56th Street, Suite 405, New York, NY 10019. *Your contribution less \$275 per ticket is tax-deductible.*

NAME: _____

COMPANY: _____

ADDRESS: _____

CITY: _____

STATE: ____ ZIP: _____

E-MAIL: _____

PHONE: _____

FAX: _____

CREDIT CARD: ☐ Visa ☐ MasterCard ☐ AmEx

CARD NUMBER: _____

EXP. DATE: _____

CARDHOLDER SIGNATURE: _____

TOTAL \$ _____

For further information, please contact Debbie Fife: Phone: 212-245-6570 ext. 20 | Fax: 212-581-8717 E-mail: dubinbreastcenter@eventassociatesinc.com Website: www.dubinbreastcenter.org

DUBIN BREAST CENTER BENEFIT COMMITTEE: PAULINE DANA AND RAFFI ARSLANIAN · MICHELE AND TIMOTHY BARAKETT · LISA AND JEFF BLAU · ANN COLLEY · JULIE ANNE QUAY AND MATTHEW EDMONDS · LISE AND MICHAEL EVANS · EILEEN PRICE FARBMAN AND STEVEN FARBMAN · TANIA AND BRIAN HIGGINS · LAURA KRUPINSKI · MARCY AND MICHAEL LEHRMAN · CHRISTINE MACK · ALICE AND LORNE MICHAELS · THALIA AND TOMMY MOTTOLA · DORE HAMMOND AND JAMES NORMILE · ANN

O'MALLEY · TRISH PALIOTTA · BETH AND JASON ROSENTHAL · CAROLYN AND CURTIS SCHENKER ·
LESLEY AND DAVID SCHULHOF · LYNN AND STEPHAN SOLOMON

FOR FURTHER INFORMATION, CALL **212-245-6570**

DUBINBREASTCENTER@EVENTASSOCIATESINC.COM WWW.DUBINBREASTCENTER.ORG

Recovery Method — Technical Details

The Problem

EFTA00400459.pdf is a 76-page scanned document from the Epstein DOJ Dataset 9. The DOJ printed the original email (which contained a MIME base64-encoded PDF attachment), then scanned it back as a PDF image with an OCR text layer. The OCR text layer contains the base64 data, but with significant character-level errors introduced by OCR misreading the Courier New monospace font.

Root cause: Courier New renders 1, l, and I nearly identically. Same for 0 and O. The OCR engine also inserted spurious characters (., ,, (, -, etc.) and frequently miscounted character widths, producing lines that were too long or too short.

What Failed (19 Approaches)

#	Approach	Result
1	Strip invalid chars from original OCR	Misaligns byte boundaries
2	Substitute common OCR errors	Makes corruption worse
3	Brute-force character scoring	Combinatorial explosion
4	qpdf repair on decoded PDF	Cannot fix stream-level corruption
5	pikepdf repair	Same — structural repair can't fix byte errors
6	Ghostscript render	Crashes on corrupt streams
7	mutool clean	Cannot repair
8	pdfimages extract	No embedded images in the decoded PDF
9	pdftoppm render	Fails on corrupt streams
10	pdftotext extract	No text extractable from corrupt streams
11	XMP thumbnail extract	No thumbnail embedded
12	Exhaustive zlib scan across raw bytes	No valid zlib headers found
13	Per-line decode of original OCR text	276,024 bytes, correct header, 0/40 streams decompress
14	OCR error correction + brute-force zlib	23-45% corruption per stream, too deep
15	inflateSync (zlib sync point recovery)	No flush points in Adobe CS4 FlateDecode
16	DEFLATE sync point scanning (academic method)	Only found garbage, no recoverable PDF content
17	Tesseract re-OCR with base64 char whitelist	WORSE: 9% good lines vs 65% original
18	KoKuToru templates on wrong scan resolution	2% byte match (wrong templates for our images)
19	Partial zlib decompression attempts	0 bytes recovered from any stream

How to Reproduce This Recovery (Step-by-Step)

If you want to independently verify or reproduce this recovery, follow these instructions exactly.

Prerequisites

Operating System: macOS, Linux, or Windows (WSL)
 Python: 3.8+
 Storage: ~500 MB free space

Install dependencies:

```
# System packages (macOS with Homebrew)
brew install poppler      # provides pdfimages

# Python packages
pip install torch torchvision Pillow
```

If on macOS — you need a case-sensitive filesystem because the KoKuToru templates have filenames like `letter_A_0.png` and `letter_a_0.png` which collide on macOS's default case-insensitive HFS+/APFS. Linux users can skip this.

```
hdiutil create -size 50m -fs "Case-sensitive APFS" \
  -volname CaseSensitive casesensitive.dmg
hdiutil attach casesensitive.dmg
# Working directory: /Volumes/CaseSensitive/
```

Step 1: Obtain EFTA00400459.pdf

Download EFTA00400459.pdf from Epstein DOJ Dataset 9. This is the 76-page scanned email document (11.25 MB). Verify:

```
$ file EFTA00400459.pdf
EFTA00400459.pdf: PDF document, version 1.6

$ ls -la EFTA00400459.pdf
# Should be approximately 11,796,482 bytes (11.25 MB)

$ pdftinfo EFTA00400459.pdf
Pages: 76
```

Step 2: Extract Raw Page Images

```
mkdir -p pdfimages_out
pdfimages -png EFTA00400459.pdf pdfimages_out/img
```

This produces 76 PNG files: `img-000.png` through `img-075.png`.

Verify the images:

```
$ file pdfimages_out/img-000.png
img-000.png: PNG image data, 816 x 1056, 8-bit grayscale, non-interlaced
```

```
$ ls pdfimages_out/ | wc -l
76
```

- `img-000.png` = email header page (NOT base64 — skip this)
- `img-001.png` through `img-075.png` = base64 content pages

Step 3: Clone and Set Up KoKuToru Template-Matching OCR

```
# On macOS, clone to case-sensitive volume:
cd /Volumes/CaseSensitive/
git clone https://github.com/KoKuToru/extract_attachment_EFTA00400459.git
cd extract_attachment_EFTA00400459

# Verify templates exist (342 PNG files in letters_done/)
ls letters_done/ | wc -l
# Should be 342
```

The repo contains:

- `ocr.py` — the template-matching OCR engine
- `letters_done/` — 342 character template PNGs (8x12 pixels each)
- Each template is named `letter_<char>_<variant>.png`

Step 4: Run Template-Matching OCR on Each Page

Copy your extracted page images into the KoKuToru directory and run the OCR:

```
# Copy base64 page images (skip img-000 which is the email header)
cp /path/to/pdfimages_out/img-001.png ... img-075.png ./

# The KoKuToru ocr.py expects images in a specific location.
# You may need to modify the input path in ocr.py, or run it per-image.
python3 ocr.py
```

How the OCR works internally:

```
import torch
from PIL import Image

# Grid parameters (tuned for this specific scan resolution)
letter_w = 8          # template width in pixels
cell_w = 7.8          # character cell width (8 - 1/5, accounts for sub-pixel drift)
letter_h = 12         # template height in pixels
line_h = 15           # line height (12 + 3 pixel spacing)
y_start = 39          # pixels from top to first text line
x_start = 61          # pixels from left to first base64 char (after "> " prefix)
```

```
# Image preprocessing: quantize pixel values to reduce scan noise
# pixel = round(pixel * 64) / 64

# For each character position in the grid:
# 1. Extract 8x12 pixel region from page image
# 2. Compute L1 loss (sum of absolute pixel differences) against all 342 templates
# 3. The template with the lowest L1 loss wins
# 4. Output that character
# Output newline every 76 characters
```

Verify your OCR output:

```
wc -l base64_extracted.txt
# Expected: ~4842

awk '{ print length }' base64_extracted.txt | sort | uniq -c | sort -rn | head
# The vast majority should be 76
```

Step 5: Handle the First Line

The first page of base64 (`img-001.png`) contains the PDF header line starting with `JVBERi0xLjU` (which decodes to `%PDF-1.5`). The KoKuToru OCR may start at line 2 because the first page also has email header text above the base64 block.

Check if the first line is present:

```
head -1 base64_extracted.txt
# Should start with JVBERi0 (= %PDF-)
# If it doesn't, you need to prepend it
```

If the first line is missing, extract it from the original OCR text layer:

```
pdftotext EFTA00400459.pdf - | grep "JVBERi0" | head -1
```

Step 6: Find and Remove the MIME Boundary

The base64 data ends before the MIME boundary. Check the end of your file:

```
tail -10 base64_extracted.txt
# Remove any lines containing _002_, cpusers, carillon, or CECCBD6
```

Step 7: Decode Base64 to PDF (Per-Line Method)

```
#!/usr/bin/env python3
import base64

VALID_B64 = set("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=")

with open("base64_extracted.txt") as f:
    lines = [line.strip() for line in f if line.strip()]

# Remove MIME boundary lines at end
while lines and any(x in lines[-1] for x in ['_002_', 'cpusers', 'carillon']):
    lines.pop()

chunks = []
good = 0
for i, line in enumerate(lines):
    cleaned = "".join(ch for ch in line if ch in VALID_B64)
    is_last = (i == len(lines) - 1)
    if is_last:
        r = len(cleaned) % 4
        if r: cleaned += "=" * (4 - r)
    try:
        chunks.append(base64.b64decode(cleaned))
        good += 1
    except:
        chunks.append(b'\x00' * 57)

result = b"".join(chunks)
print(f"Decoded: {len(result)} bytes (expected ~276,028)")
print(f"Good lines: {good}/{len(lines)}")

with open("DBC12_recovered.pdf", "wb") as f:
    f.write(result)
```

Expected output:

```
Decoded: 275971 bytes (expected ~276,028)
Good lines: 4842/4842 (100%)
PDF header: b'%PDF-1.5\r%\xe2\xe3\xcf\xd3\r\n'
```

Step 8: Validate — Decompress FlateDecode Streams

```
#!/usr/bin/env python3
import zlib

with open("DBC12_recovered.pdf", "rb") as f:
```

```

    data = f.read()

pos = 0
stream_num = 0
success = 0
while True:
    marker = data.find(b'stream', pos)
    if marker < 0: break
    cs = marker + 6
    while cs < len(data) and data[cs:cs+1] in [b'\r', b'\n']: cs += 1
    es = data.find(b'endstream', cs)
    if es < 0:
        pos = marker + 6
        continue
    sd = data[cs:es]
    stream_num += 1
    for wbits in [15, -15, 31]:
        try:
            dc = zlib.decompress(sd, wbits)
            print(f" Stream #{stream_num}: {len(dc)} bytes OK")
            success += 1
            with open(f"stream_{stream_num}.bin", "wb") as f:
                f.write(dc)
            break
        except: pass
    pos = es + 9

print(f"\nResult: {success}/{stream_num} streams decompressed")
# Expected: 39/40

```

Expected output:

```

File size: 275971 bytes
Streams: 40
  Stream #1: 300 bytes OK
  Stream #2: 1122 bytes OK
  ...
  Stream #39: 4521 bytes OK
  Stream #40: [fails – spans the corrupt first line]

Result: 39/40 streams decompressed

```

Step 9: Extract Text from Decompressed Content Streams

```

#!/usr/bin/env python3
import re, glob

```

```
def extract_text(data):
    text = data.decode('latin-1')
    result = []
    for m in re.finditer(r'\(((^)]*)\\s*Tj', text):
        result.append(m.group(1))
    for m in re.finditer(r'\[([.*?])\\s*TJ', text):
        strings = re.findall(r'\(((^)]*)\\s', m.group(1))
        result.append("".join(strings))
    return result

all_text = []
for sf in sorted(glob.glob("stream_*.bin")):
    with open(sf, "rb") as f:
        texts = extract_text(f.read())
        if texts: all_text.extend(texts)

for line in all_text:
    if line.strip(): print(line)
```

Step 10: Verify Against Known Content

Check for these key strings in the extracted text:

DUBIN BREAST CENTER
SECOND ANNUAL BENEFIT
MONDAY, DECEMBER 10, 2012
MANDARIN ORIENTAL
ELISA PORT, MD, FACS
CYNTHIA MCFADDEN
Tax-ID# 13-6171197
Event Associates, Inc.
162 West 56th Street, Suite 405
212-245-6570
dubinbreastcenter@eventassociatesinc.com

If all of these appear in your extracted text, the recovery is confirmed.